

# Base: An Interface for Programming the Board

<https://github.com/unnamedlambda/base>

## Abstract

Base is an interface and system for programming at the scope of an entire board. The CPU, GPU, network and file system are all controlled through a single artifact loaded by a driver. Its compiled entry points can then be called at run time, with the driver embedded through its C API and usable from Rust, Python, C and Lean. Lean generators for artifacts have theorems which prove, at build time, properties of the run-time behavior. Performance matches standalone Rust on CPU workloads and PyTorch on GPU workloads.

## 1 System Architecture

Modern computer applications run on many devices at once, each with its own control interface. A CPU has an instruction set, which a language like C is compiled to directly. A GPU has a driver API, reached through system software and usually programmed in a kernel language of its own such as CUDA [3]. Networks and file systems are reached by system calls the CPU makes, with values computed at run time. Each of these interfaces is precise in isolation, but real applications require them to be composed together correctly. The programmer must reason about CPU arithmetic, GPU kernel launches and their results, file paths and the contents of writes, and above all about the interleaving of data between these interfaces across the program's whole run.

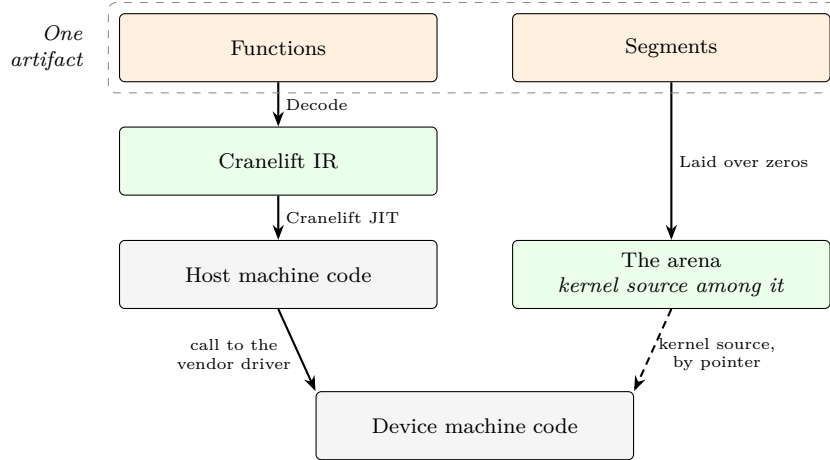
For a single general-purpose device this is solved by an instruction set: a finite, documented vocabulary through which the device's components are controlled. A program for that instruction set is a standalone object, independent of source details. Whether it was written by hand or generated by a compiler, the same final object is the whole of the interface and governs the resulting behavior. Base extends the scope of such an interface to the entire board. A software driver accepts full board control via an *artifact*: a value that can be serialized, exist in memory, and be the subject of logical reasoning.

### 1.1 Artifact

An artifact is represented as the Rust struct below and is encoded as the CBOR data format [4]. It contains both a program and the memory it starts with.

```
#[derive(Serialize, Deserialize, Debug, Clone, PartialEq)]
#[serde(deny_unknown_fields)]
pub struct Artifact {
    pub functions: Vec<clif::Function>,
    pub required_memory: u64,
    pub data: Vec<Segment>,
}
```

A user decodes a serialized artifact, loads it, and calls an entry point by name.



**Figure 1:** The functions are decoded and compiled for the host processor. The segments are laid into the arena, kernel source among them, and that source reaches the vendor’s driver.

```

let artifact = Artifact::from_bytes(ARTIFACT_BYTES)?;
let mut driver = Driver::load(artifact)?;

let status = driver.execute("infer", input, output)?;

```

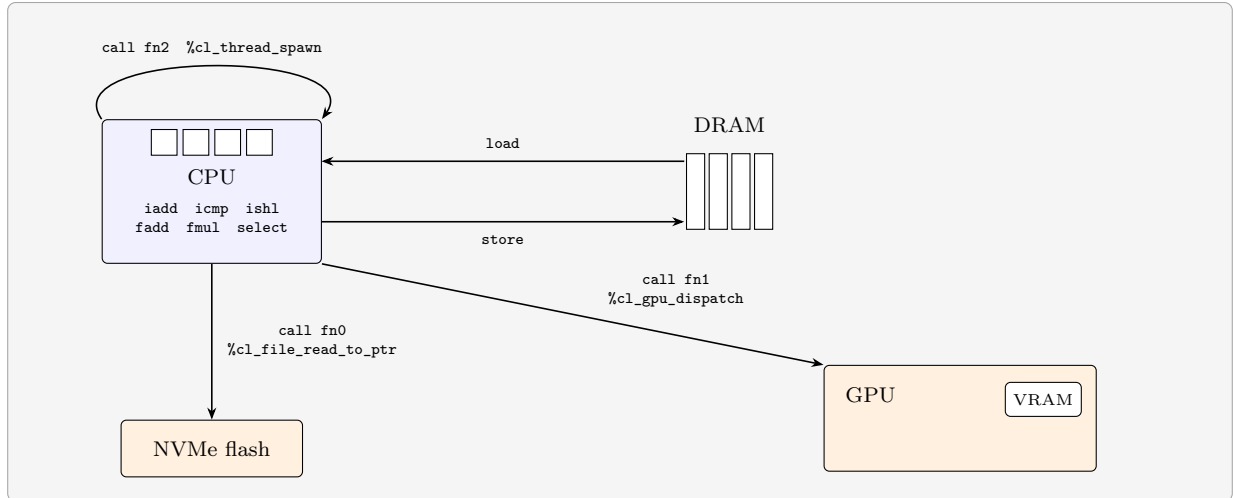
The program is CLIF, the IR of the Cranelfit code generator [1]. During load the driver JIT compiles the artifact’s program to machine code and initializes a shared arena according to the artifact’s specification. `required_memory` is how much space to reserve for the arena, and the segments of `data` are laid over it, so an artifact isn’t forced to specify every initial zero. Calls to `execute` invoke functions keyed by the names the artifact carries, with the caller supplying both the input and output pointers. The same arena is shared between calls, so state carries across them: a device handle opened by one entry point is still open for the next.

## 1.2 Driver

The CLIF program is a list of functions, each carrying an optional name for use in `execute`. A function’s blocks, block parameters and instructions are encoded as data rather than as raw text. The driver walks that structure through Cranelfit’s instruction builder to produce machine code. Only a subset of CLIF is included: eight types, the widest a 128-bit vector, and forty-five instruction forms spanning arithmetic, comparison, bit manipulation, conversion, memory access, calls and structured control flow.

A CLIF program controls one core and the memory it addresses, and reaches for *primitives* to access the rest of the board. A primitive is a Rust function using the C calling convention, compiled alongside the driver as part of the Base crate. When an artifact is loaded, the driver hands Cranelfit each primitive’s address to enable direct calls into these functions.

Device kernels can be embedded into the artifact as source text in its data segments. The `cl_gpu_init` primitive leaves a device context at a pointer into the arena; `cl_gpu_create_pipeline` takes kernel source out of the arena and gives it to the vendor’s compiler; `cl_gpu_dispatch` launches the result, and `cl_gpu_download_ptr` writes what it computed back into the arena. Figure 1 follows both paths out of the artifact. Kernel source and vendor driver use are handled entirely this way; Base has no built-in device handling outside what a program reaches through these primitives.



**Figure 2:** A system board along with how its capabilities are accessed by CLIF. Arithmetic and memory access are instructions such as `iadd`, loads and stores. Everything else is reached by a call to a board primitive.

```
sig0 = (i64, i64, i64, i64) -> i64 system_v
fn0 = %cl_file_read_to_ptr sig0

v5 = call fn0(v1, v2, v3, v4)
```

**Figure 3:** Cranelift makes a call to `cl_file_read_to_ptr` just as it would any other function. The CLIF text format is shown for clarity.

Figure 2 shows how CLIF and the primitives control board behavior. An exploration of a specific primitive, for reading a file into a buffer, follows.

```
pub(crate) unsafe extern "C" fn cl_file_read_to_ptr(
    path_ptr: *const u8,
    dst_ptr: *mut u8,
    file_offset: i64,
    size: i64,
) -> i64 { ... }
```

It takes a NUL-terminated path and a destination to write the file contents to, and returns the count read, or `-1` to indicate a failure. The pointers are plain addresses; for a CLIF program they point into the arena, the only memory it has. The call itself is unremarkable: it sits among the arithmetic that computed its arguments, and what it returns is a value like any other (Figure 3).

## 1.3 C API

The entire Base driver is exported as a C API, shown below. This enables the driver to be embedded for calling from other languages, as C is the common foreign function interface. Python and Lean APIs are built on top of this.

```
#[no_mangle]
pub unsafe extern "C" fn base_driver_load(
    artifact: *const u8, len: usize) -> *mut Driver { ... }

#[no_mangle]
pub unsafe extern "C" fn base_driver_execute(
    driver: *mut Driver,
    name: *const u8, name_len:  usize,
    input: *const u8, input_len:  usize,
    output: *mut u8,  output_len: usize,
    status: *mut i64) -> i32 { ... }

#[no_mangle]
pub unsafe extern "C" fn base_driver_memory(
    driver: *const Driver, len: *mut usize) -> *const u8 { ... }

#[no_mangle]
pub unsafe extern "C" fn base_driver_free(driver: *mut Driver) { ... }

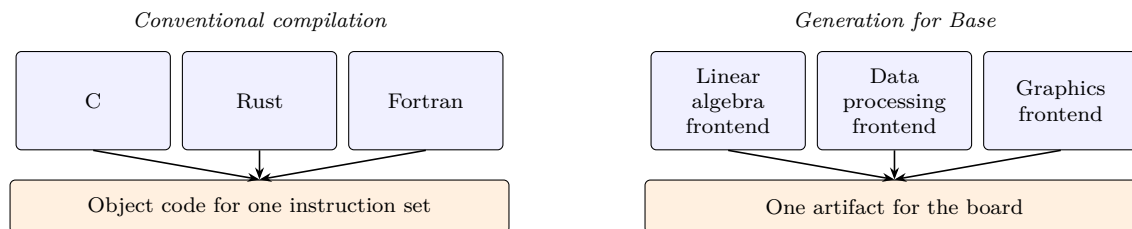
#[no_mangle]
pub unsafe extern "C" fn base_last_error(buf: *mut u8, cap: usize) -> usize { ... }
```

## 2 Programming Experience

The driver’s capabilities are accessed through the artifact, so it is clear why the construction of an artifact must be done carefully. CLIF’s instructions and control-flow features have documented behavior, which enables reasoning about correctness.

### 2.1 Artifact Generation

Artifacts are produced programmatically, and a system that does this is called a *generator*. A generator relates to an artifact much as a C compiler relates to its object code. Figure 4 puts this relation side by side.



**Figure 4:** On the left source languages are compiled into native instructions for the processor being targeted. On the right Base enables generation of artifacts from different frontends.

Lean [7] serves as the interface from which artifacts are generated. Any language or proof assistant which is capable of accessing Base through the C API is a valid candidate for a generator, but no alternative has been implemented. Artifact contents are Lean terms, serialized into CBOR and used with the driver interface.

The program below replaces every NUL byte in a buffer with a space. Lean’s `do` notation over a `Prog` term binds values much as an IR builder does in an application with an embedded compiler. The Lean here runs to produce the artifact value, which is what performs the byte replacement.

```
def blend (v nul space : V .i8x16) : Prog V L (V .i8x16) := do
  bitselect (← icmp .eq v nul) space v

def scrub (vectors : Nat) (src dst : V .i64) : Prog V L Unit := do
  let nul ← splat .i8x16 (← iconst .i8 0)
  let space ← splat .i8x16 (← iconst .i8 32)
  forLoop (← iconst64 vectors) fun i => do
    let off ← ishlImm i 4
    let v ← loadI8x16 (← iadd src off)
    store (← blend v nul space) (← iadd dst off)

def code : Prog V L Unit := do
  scrub 256 (← dataPtr) (← outPtr)
```

Figure 5 follows three lines of the scrub loop from Lean through CLIF to the machine code Cranelift emits.

| <i>Lean, by the user</i>                                                  | <i>CLIF, in the artifact</i>                                                            | <i>x86-64, for the host</i>                                                                    |
|---------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <code>loadI8x16 (← iadd src off)</code>                                   | <code>v16 = iadd.i64 v1, v15</code><br><code>v17 = load.i8x16 notrap aligned v16</code> | <code>vmovdqu (%rsi, %rdi), %xmm2</code>                                                       |
| <code>bitselect (← icmp .eq v nul)</code><br><code>space v</code>         | <code>v18 = icmp eq v17, v8</code><br><code>v19 = bitselect.i8x16 v18, v10, v17</code>  | <code>vpcmpeqb %xmm3, %xmm2, %xmm3</code><br><code>vpblendvb %xmm3, %xmm4, %xmm2, %xmm2</code> |
| <code>store (← blend v nul space)</code><br><code>(← iadd dst off)</code> | <code>v20 = iadd.i64 v3, v15</code><br><code>store notrap aligned v19, v20</code>       | <code>vmovdqu %xmm2, (%rcx, %rdi)</code>                                                       |

**Figure 5:** Three representations of the same computation as handled by Base.

## 2.2 Reasoning About Emitted Code

The intention of the scrub code above is to leave no NUL bytes in the buffer. Below is a statement about the emitted instructions which are generated for the artifact. Each input byte is confirmed to carry through to the output, with the desired substitution performed.

```

def Answers (Γ : Env) (c : Code) (binds : Nat) (v : V) : Prop :=
  ∀ (fuel : Nat) (cfg : Cfg) (w : World), ∃ Γ',
    runCode (fuel + 2) cfg Γ w c = .ok Γ' w
    ∧ Γ'.size = Γ.size + binds
    ∧ Γ'.back? = some v

theorem blend_leaves_no_nul
  (Γ : Env) (tys : List ClifTy) (hΓ : tys.length = Γ.size)
  (a b c : R) (xs : Array UInt64) (h16 : xs.size = 16)
  (ha : Γ[a]? = some (.vec .i8x16 xs))
  (hb : Γ[b]? = some (.vec .i8x16 (.replicate 16 0)))
  (hc : Γ[c]? = some (.vec .i8x16 (.replicate 16 32))) :
  ∃ ys, ys = xs.map (fun x => if x = 0 then 32 else x)
    ∧ Answers Γ (Prog.emit (discard (blend a b c)) tys) 2 (.vec .i8x16 ys)
    ∧ ∀ y ∈ ys, y ≠ 0 :=
  ⟨scrubLanes xs, rfl,
  fun fuel cfg w =>
    ⟨_, blend_scrubs fuel cfg w Γ tys hΓ a b c xs h16 ha hb hc, by simp,
    by simp [scrubbedBytes]⟩,
  scrubLanes_no_nul xs⟩

```

The theorem above does not cover the splats or any behavior of the loop around the blend.

## 2.3 Accessing the Board

A program that drives other device capabilities besides the CPU is written with the same Lean interface. The following example renders a 4096×4096 Mandelbrot set on the GPU and writes it to a bitmap file. The artifact's data segments hold the GPU kernel source (the shader) and other values used at run time.

```

def code : Prog V L Unit := do
  let ptr ← basePtr
  gpuInit ptr
  let dataSz ← iconst64 pixelBytes
  let bufId ← gpuCreateBuffer ptr dataSz
  let shOff ← fldOffset f.shader
  let bdOff ← fldOffset f.bindDesc
  let one32 ← iconst32 1
  let pipeId ← gpuCreatePipeline ptr shOff bdOff one32
  let wgx ← iconst32 wgX
  let wgy ← iconst32 wgY
  let _ ← gpuDispatch ptr pipeId wgx wgy one32
  let pxOff ← fldOffset f.pixels
  let _ ← gpuDownload ptr bufId pxOff dataSz
  gpuCleanup ptr
  let total ← iconst64 (54 + pixelBytes)
  let _ ← fldWriteFile0 ptr f.filename f.bmpHeader total

```

### 3 Evaluation

Base has its speed evaluated on workloads to ensure it can maintain the speeds that are achievable with systems languages. The comparison is made with standalone Rust [5] compiled through LLVM [2] implementing the same workload. To assess device work, GPU workloads in Base are compared with PyTorch [6].

**Table 1:** A comparison for workloads written in idiomatic low-level Rust, Base using only CLIF, and Base generating native assembly, on an AMD Ryzen 5 5500.

| Workload        | Rust ( $\mu$ s) | CLIF / Rust | asm / Rust | Cause                 |
|-----------------|-----------------|-------------|------------|-----------------------|
| Mandelbrot set  | 1800.4          | 1.00        |            |                       |
| Pointer chase   | 2295.5          | 1.00        |            |                       |
| Byte histogram  | 320.5           | 1.00        |            |                       |
| Polynomial, f32 | 6.1             | 2.04        | 1.01       | 128-bit vectors       |
| Copy, 32 MB     | 2296.8          | 1.56        | 1.00       | no non-temporal store |

#### 3.1 CPU Workloads

Base with CLIF is level with Rust for the first three workloads. The other two see Rust accessing capabilities that CLIF lacks support for: 256-bit vectors for the polynomial and non-temporal stores for the copy. To close this gap, Base uses native assembly, assembled in Lean and accessed using the `cl_native_load` primitive.

#### 3.2 GPU Workloads

On GPU workloads Base is comparable to PyTorch in whichever of eager and compiled mode is faster. Small workloads are faster in Base, where fusion and a lower cost of dispatch improve over PyTorch.

**Table 2:** GPU workload comparisons on an RTX 3060.

| Workload                    | PyTorch ( $\mu$ s) | Base / PyTorch |
|-----------------------------|--------------------|----------------|
| Vector add 50M              | 1810               | 0.99           |
| Scale-and-add 50M           | 1860               | 0.97           |
| Scale-and-add 1M            | 90                 | 0.52           |
| GEMV $11008 \times 4096$    | 555                | 0.98           |
| RMSNorm 4096                | 57                 | 0.19           |
| Softmax 32000               | 30                 | 0.37           |
| Decode attention, 2048 keys | 83                 | 0.77           |
| Decode attention, 128 keys  | 67                 | 0.22           |

## 4 Limitations

While Lean enables mechanical reasoning over the artifact and the code used to generate it, the components of the infrastructure that it cannot cover are worth considering. The model of CLIF is translated from the documentation and tested against Cranelift on a corpus of 482 cases. The driver and the primitives are implemented and tested in Rust, and no Lean proof can reach into these implementations; it can only state assumptions about them, which can be reviewed by hand.

Not every application can accommodate embedding the Base driver along with the primitive implementations. Base is portable across x86-64 and ARM and uses no features specific to one operating system, but platforms outside these are not a fit, nor are applications that cannot accept the driver’s cost at load: the most expensive artifact in the repository takes 45 ms to decode and 600 ms to compile.

## 5 Conclusion

Base enables programming at the scope of an entire system board. A single interface accepts CPU control via CLIF programs, a shared memory arena, and a selection of primitive functions CLIF can call, which cover device, network and other board behavior. The artifact and driver API allow Base to be embedded both in systems code, which interacts with existing artifacts, and in proof assistants, which generate artifacts and invoke the driver through its C API. A Lean codebase allows programming applications at native speed while embedding proofs about their behavior.

## References

- [1] Bytecode Alliance. *Cranelift: A Code Generation Toolkit*. <https://cranelift.dev/>.
- [2] Chris Lattner and Vikram Adve. *LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation*. CGO, 2004.
- [3] NVIDIA. *CUDA C++ Programming Guide*. <https://docs.nvidia.com/cuda/>.
- [4] Carsten Bormann and Paul Hoffman. *Concise Binary Object Representation (CBOR)*. RFC 8949, IETF, 2020.
- [5] Nicholas D. Matsakis and Felix S. Klock II. *The Rust Language*. HILT, 2014.
- [6] Adam Paszke et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. NeurIPS, 2019.
- [7] Leonardo de Moura and Sebastian Ullrich. *The Lean 4 Theorem Prover and Programming Language*. CADE, 2021.